1305ENG: Engineering Programming

Programming Laboratory 2: Variables, Operators, and Conditionals

Instructions:

This lab will extend your knowledge of the C language, introducing variables and the basic control structures which are conditional statements (if and switch). It will also give you more practice at the things you learned last week, such as printf and scanf statements and using library functions.

You should have read all "Operators, Flow Control and Data Types" notes and watched all of the associated videos prior to attending this lab!

Homework Questions:

These questions should be completed *prior* to the lab session.

1. **Write a C program which declares 4 integer variables a, b, c, and d. a and b should be assigned the values 6 and 13 respectively *when they are declared*. The variable c should be read from the keyboard, and d should then be set as the sum of a, b, and c, then printed to the screen.**
2. **Write a C program that declares two integer variables *a* and *b*, and a floating point variable *f*. Assign a and b the values 7 and 2 respectively. Divide a by b and store the result in *f*, then print *f* to the screen.**
3. **Write a C program that reads two integers from the keyboard, stores them in variables, then prints out their sum.**
4. **Write a C program that reads two integers from the keyboard, stores them in variables, then prints out the *remainder* of the first divided by the second.**
5. **Write a C program that reads a single integer from the keyboard, then prints out "Big" if the number is greater than 10, or "Little" otherwise.**
6. **Write a C program that reads a single integer from the keyboard, then displays whether the number is even or odd.**

PART 1: Variables, Types, and Operators

From last week, you will recall that to declare variables, you have a line in the program which contains the *type* of the variable followed by one or more variable names, separated by commas. Variables can be given an initial value by following the variable name with and equals sign and the value you'd like to give it. For example, the following code will create two integer variables (*a* and *b*), one floating point variable *c* with the initial value 3.14, and a char variable *d* with the value 'f'. Note that the letter f must be surrounded by *single quotes*, which is always used to represent a single character. This tells the compiler to replace that character with the ASCII value of 'f', which is 102.

```
int a, b ;
float c = 3.14 ;
char d = 'f' ;
```

Operating on variables in C follows strict order of precedence. These are very similar to mathematical precedence, however there are some differences. The assignment (=) operator has the *lowest* precedence, so will always operate last. As each operator also works independently of others in the same statement, this can have some interesting effects. To demonstrate this, perform the following operations:

1. **Type the section of code above into a .c file, within the normal main() function just like the programs you wrote last week. As usual put the #include <stdio.h> directive at the top as well.**
2. **Add lines in this program so that the variables *a* and *b* are given the values of 10 and 3 respectively.**
3. **Divide *a* by *b* and store the result in *c*.**
4. **Use printf to display the value of *c* to the screen in an appropriate format.**
5. **Compile and run this program, and comment on the result. Is it what you expected?**
6. **Change your program again so that *a* and *b* are read from the keyboard instead of being set directly. Run this new program many times with different values, and check the results. Can you now see what is happening?**
7. **How can you fix this problem so that the result is 'correct'? Discuss this with each other and/or the demonstrators to come up with a solution, then modify your program to test.**

Operators

The C language contains many operators, which will be discussed in the lectures. We will cover some of the more interesting and useful operators in this lab.

The standard mathematical operators are all available in C. There are also some more interesting operators that are very useful in programming, such as the % (remainder or modulus) operator. Here is a non-complete list of mathematical operators in C:

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder (valid for integer types only)
++	Increment
--	Decrement

The first four should be very obvious. However the way they function also sometimes depends on the type of variables that they are used with. In general, the following rules will apply:

- If both operands are of the same type, they will not be changed and the operation will produce a value of the same type.
- If the operands are of different types, the 'smaller' type will be converted to be the same as the bigger, and the result will be of the bigger type. A char is the smallest type, followed by int, float, and double the biggest.
- Unary (single operand) operators have the highest precedence, from right to left
- Multiplication, division, and modulus (%) have next highest priority, from left to right
- Addition and subtraction are next highest, left to right

- Bitwise operators (covered later) have next highest precedence
- Other C operators (relational, logical, and assignment) are all of lower precedence, and will be covered later.
- Normal brackets ( ) can be used to raise precedence just like in mathematics.

Complete the following exercises, which will give you practice with a few common operators.

8. **Create a new C program (with the usual pre-processor directives and main function) that creates two integer variables, reads their values from the keyboard using scanf, then calculates their sum (+), difference (-), product (*), quotient (/), and remainder (%). You can either store these 5 values in new variables, or print then directly to the screen. Make sure you use user-friendly output statements, such as:**

```
Enter two numbers: 7 2
Sum: 9
Difference: 5
Product: 14
Quotient: 3
Remainder: 1
```

9. **Complete assessable question 1 below.**

PART 2: Conditional (if and case) Statements

Up until now, all of the programs we have written have been very simple sequences of instructions, each of which is executed in turn until the end of the program. This is only possible for extremely simple tasks, the vast majority of programs need to change their behaviour depending on the data to be processed. The simplest mechanism for controlling the flow of a program are conditional statements, which decide whether to perform an action or not based on a condition that may or may not be true. There are two types of conditional statements in C, the ***if..else*** statement, and the ***switch..case*** statement.

The ***if..else*** statement is the simplest of these, and operates by checking if a condition is true or false. If it is true, then a block of code will be executed. If it is false, then nothing is done, or optionally a second block of code can be executed (the *else* block). Here is a very simple example of an if statement:

```
#include <stdio.h>

int main ( void )
{
     int a ;

     printf ( "Please enter an integer value: " ) ;
     scanf ( "%d", &a ) ;

     if ( a >= 0 )
          printf ( "You entered a positive value!\n" ) ;
```

```
    else
        printf ( "You entered a negative value.\n" ) ;
}
```

1. **Type this program into an editor, compile and run it, and make sure it gives the expected results.**

This program checks if the number entered is positive or negative (zero is considered positive for this case) and prints that to the screen. The *condition* part of the statement (a >= 0), is very typical, and relies on another type of C operator, relational operators. These are once again very similar to the corresponding mathematical operators, and are as follows:

==	Equal (note there are two equal signs, not one!)
!=	Not equal
>	Greater than
>=	Greater than or equal
<	Less than
<=	Less than or equal

As mentioned previously, the precedence of these operators is lower than the other mathematical operators, so any expressions to the left or right will be evaluated before they are compared.

The C language also has a somewhat interesting way of representing true and false. False is always represented by the numerical value zero (0). However *any other value* is considered TRUE, regardless of whether it is positive, negative, or how large or big it is. So while it might not make much sense to a human, the statement:

```
if ( 7 )
```

is perfectly valid in C, and will be considered TRUE.

When evaluating relational operators, however, the 'result' of the operator will always be 0 (false) or 1 (true). For example, `5 == 5` will evaluate to be 1, `3 > 6` will evaluate to 0, and so on. You can test yourself by writing a C program and using printf to display the results of relational operators.

So, you've now used an if statement and relational operators. One thing you might have noticed was that the *body* of the if statement (the printf statements) were only a single statement. By default, an *if* statement can only have a single statement within it. To include more than one statement in the body, you need to enclose all the statements in braces { }. Just like the braces surrounding the contents of the function, anything within these braces is considered as a single entity, belonging to the *if* or *else* section. The general form of an *if..else* statement is thus:

```
if ( condition ){
    Code performed if condition is true ;
    And some more code as well ;
} else {
```

```
        Code performed if the condition is not true ;
        More statements ;
}
```

Because it's easier to always write code in the way, it is strongly recommended that you ***always use the braces, even if there is only one statement in the body!***

Nested if..else Statements

Often, you would like to check multiple things at the same time. In the program above that checked for positive or negative numbers, for example, it would be nice to check for the zero condition as well. This is possible by *nesting* the *if..else* statements, which allows for multiple conditions to be checked.

2. **Modify the example code from above as follows, then compile and run it, checking that it works for various input values.**

```
if ( a > 0 ){
     printf ( "You entered a positive value!\n" ) ;
} else if ( a == 0 ){
     printf ( "You entered zero.\n" ) ;
} else {
     printf ( "You entered a negative value.\n" ) ;
}
```

The braces aren't necessary in this example, since there is only one statement within each section, however it is good to include them anyway for consistency.

You can have as many *else if* sections as necessary in this statement, checking for many different things. However at most only ***one*** section will be executed – once one condition is found to be true, that section executes and the statement finishes. If you need multiple things checked for of which many can be true, you should use separate if statements instead.

Logical Operators

Often, you will need to check if more than one condition is true (or false). To do this in C, you can combine relational operators with *logical operators*. The three logical operators that we will use with *if* statements are NOT (!), OR (||), and AND (&&). Using NOT will reverse a logical value (true becomes false, false becomes true), OR will combine two logical values and be true if *either* is true, while AND combines two and is only true if *both* are true.

Consider the simple task of asking a person to enter a number between 1 and 10. We will need to check that the number entered is both greater than or equal to 1, AND less than or equal to 10. Clearly an AND operator is required here. The appropriate C code would thus be:

```
int number ;

scanf ( "%d", &number ) ;
```

```
if ( number >= 1 && number <= 10 ){
     printf ( "Your number is in range!\n" ) ;
} else {
     printf ( "That number is not between 1 and 10!\n" ) ;
}
```

Note that we could also have checked the opposite case, meaning the number is outside the range of 1..10. In this case we would check that the number is less than 1 OR greater than 10, and reverse the if and else sections. These two sections of code are entirely equivalent.

```
int number ;

scanf ( "%d", &number ) ;
if ( number < 1 || number > 10 ){
     printf ( "That number is not between 1 and 10!\n" ) ;
} else {
     printf ( "Your number is in range!\n" ) ;
}
```

3. **Type the two sections of code above into a C program and ensure that they both work as intended.**

Switch..Case Statements

The second type of conditional statement in C is the switch..case statement. This is not nearly as common as the if statement, and can only be used in very specific situations. Note that it is always possible to use an if statement instead of a switch..case statement, but in some situations the switch..case may be more efficient.

The exact syntax of the switch..case statement is as follows:

```
switch ( variable ){
     case val1:     statement1 ;
                    statement2 ;
                    break ;
     case val2:     statement1 ;
                    statement2 ;
                    statement3 ;
                    break ;
     …
     default:       statement1 ;
                    statement2 ;
                    statement3 ;
}
```

When executing this code, the value of *variable* is compared to each *case* value, and if one matches the corresponding statements are executed, up until the *break* line. If none of them match, the *default* section (if it exists, it is optional) is executed instead. There are a few things to note about these statements though:

- The *case* values must be **literal** values. That means they must be actual numbers (like 3, -5, 0, etc) or characters ('a', '?', '\n'), they cannot be variables.
- The switch variables must *exactly* match the case value. You cannot have greater than, less than, etc.
- You must have a break statement at the end of each case block. If you do not, execution will continue into the next case block until break is found.
- There are no braces around the case blocks, as there are in most C syntax. The break statement determines where the block stops.
- The default section is optional. If there is none, and the variable does not match any of the case values, nothing is done.

Because of the restrictions of switch..case statements, they are only usable in a few situations, and are quite rare in C programs in general. In general, they are used when a variable can only have a few specific values, and you'd like to check for each. One great example of this is a menu, where the user enters a choice and it must be one of only a few options.

4. **Type in the following example of a switch..case statement, compile and run it.**

```
#include <stdio.h>

int main () {

   char grade ;
   printf ( "Please enter your grade (A-E): " ) ;
   scanf ( " %c", &grade ) ;

   switch(grade) {
      case 'A' :
         printf("Excellent!\n" );
         break;
      case 'B' :
         printf("Well done!\n" );
         break;
      case 'C' :
         printf("You passed\n" );
         break;
      case 'D' :
         printf("You can do better\n" );
         break;
      case 'F' :
         printf("Better try again\n" );
         break;
      default :
         printf("Invalid grade\n" );
   }

   printf("Your grade is %c\n", grade );
```

```
    return 0;
}
```

Assessable questions:

You may be called to present one of these questions during the last hour of the session, and this presentation will count for 5% of your final grade for the course. You will be called to do two such presentations during the trimester, at random times. Note that you will need to explain your solution to the demonstrator and make changes to the code if requested to demonstrate your understanding. Each question should take less than 5 minutes to complete, and you may be asked a few simple questions while presenting. If you are not present in the laboratory when you are due to present, you will receive 0 for this assessment item.

1. **Create a new C program called resist.c which prompts the user to enter two resistor values (floats). Your program should then calculate the series and parallel combination of these resistors and print the result to the screen. For reference, resistors in series are simply added (R1+R2) and parallel resistance is calculated by (R1*R2)/(R1+R2).**

2. **Write a C program that asks the user to enter two integer values. Your program should then check if the first number is divisible by the second number (use the remainder % operator for this). If it is, the text "x is a multiple of y" should be displayed (where x and y are the two numbers). If not, the remainder should be displayed instead. For example:**

   ```
   Please enter two integers: 16 3

   16 is not a multiple of 3, the remainder is 1.
   ```

   Hint: it will be helpful to calculate the remainder first and store it in a variable. Then use that variable to check if the numbers are divisible. The variable can also then be displayed later if necessary.

3. **Write a C program which prompts the user to enter a letter (a-z). Your program should then check that it is a letter (Hint: check that it is between the letters 'a' and 'z' inclusive), and also that it is not the letter 'x', which is not allowed. If the letter is valid print "valid letter" to the screen, otherwise print "invalid letter". Compile and test your program to ensure that it works correctly.**

4. **Write a C program that allows the user to choose between various arithmetic operations on two numbers. The program should display a menu with options to perform the following operations:**

   a. **Addition**
   b. **Subtraction**
   c. **Multiplication**
   d. **Division (with a check to prevent division by zero)**

   It should then read the two operands, perform the requested operations, and display the result. Use a switch-case statement to manage the menu selection. For division, include an if-else statement to check for division by zero and prompt the user accordingly.

(2 marks)

Extra practice questions (complete in your own time):

1. Modify your resist.c program from q1 so that rather than calculating both series and parallel, your program prompts the user as to which should be calculated. The user should enter a single character, either 's' for series or 'p' for parallel, with your program then calculating the displaying the appropriate result. Note that reading a character with "%c" will read the *very next character*, regardless of whether that is a space, newline character, or anything else. To prevent reading unwanted whitespace, you can read a character with " %c" (note the space in front of the %c) instead. This will remove *all* whitespace from the input stream, and read the next non-whitespace character.

2. The legal alcohol limit for driving depends upon the type of license that a driver holds. If the user has a provisional license, any amount of alcohol is considered illegal. For the purposes of testing this means a blood alcohol content (BAC) of more than 0.01%. If the driver has an open license, the driver must have a BAC no higher than 0.05%. Write a C program which prompts the user to enter the type of license they have ('p' for provisional or 'o' for open) and their blood alcohol content, and then calculate and display whether they are legally allowed to drive a vehicle or not.

3. Grades at Griffith are within the range of 1-7, and are calculated based on the percentage a student achieves in their assessment. Typically, these are assigned as follows:

   7: $85 <= x <= 100$
   6: $75 <= x < 85$
   5: $65 <= x < 75$
   4: $50 <= x < 65$
   3: $45 <= x < 50$
   2: $20 <= x < 45$
   1: $0 <= x < 20$

   Write a C program which prompts the user to enter their percentage grade (a float), and then calculates what their final grade will be and displays it to the screen. Your program should also ensure that the mark entered is in the range 0-100, otherwise and error should be displayed.

4. Write a simple guessing game in C. This game should randomly generate a number between 1 and 10 which the user must try to guess. The user should then be prompted to enter a guess (between 1 and 10), and if it is equal to the random number they win, if not they lose. If the number is out of range, they should be informed of this as well. Note that you can generate the random number using the *rand()* function, with the following lines at the start of your program. You will also need to #include <stdlib.h> and <time.h>

```
int num ;
srand ( time ( NULL )) ;
num = rand() % 10 + 1 ;
```